

Automatyzacje i integracje urządzeń w Home Assistant

Poradnik od podstaw do zaawansowanych przepisów: jak Home Assistant jest zbudowany, jakimi protokołami rozmawia z urządzeniami, jak pisać niezawodne automatyzacje — z wyjaśnieniami technicznymi dla ciekawych

wersja 2 — 7 lipca 2026 — dokument wewnętrzny projektu AMPERE_POINT

Uzupełnia: AMPERE_POINT_przewodnik_HA_instalacja_integracja_v2 (instalacja i konfiguracja od zera)

oraz AMPERE_POINT_home_assistant_xtend_tuya_raport_v1 (łańcuch Tuya w szczegółach).

Spis treści

1 Wstęp — co jest w tym poradniku i jak go czytać	4
2 Jak Home Assistant jest zbudowany	4
2.1 Encja, stan, zdarzenie — trzy pojęcia, na których stoi wszystko	4
2.2 Skąd HA pamięta historię: recorder i baza danych	5
2.3 Integracje i wpisy konfiguracyjne	5
2.4 Warianty instalacji — przypomnienie w jednym akapicie	6
3 Jak urządzenia trafiają do Home Assistant	6
3.1 Odkrywanie: skąd HA wie, że coś jest w sieci	6
3.2 Cztery klasy łączności: lokalnie/chmurowo × push/poll	6
3.3 Obie decyzje na jednym rysunku	7
4 Protokoły — jak naprawdę rozmawia się z urządzeniami	7
4.1 Wi-Fi i sieć domowa: HTTP, REST, WebSocket	7
4.2 MQTT — wspólny język automatyki	8
4.3 Zigbee — radiowa siatka czujników	9
4.4 Z-Wave — siatka na osobnej częstotliwości	9
4.5 Thread i Matter — nowy porządek	10
4.6 Bluetooth Low Energy	10
4.7 Kamery: RTSP i ONVIF	10
4.8 Modbus — język przemysłu (i falowników)	11
4.9 Ściąga porównawcza	11
5 Ekosystemy w praktyce — czym i jak łączyć konkretne urządzenia	11
5.1 Urządzenia Tuya (w tym ładowarki AMPERE POINT)	11

5.1.1	Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt	12
5.2	ESPHome — własne urządzenia w godzinę	13
5.2.1	Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt	13
5.3	Shelly i podobne: Wi-Fi, ale lokalnie	14
5.3.1	Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt	14
5.4	Telefon jako urządzenie: aplikacja Companion	15
5.4.1	Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt	15
5.5	Co wybierać na przyszłość — krótki kompas	16
5.5.1	Ścieżki wdrożenia dla kierunków z kompasu	16
6	Klik po kliku: wdrażanie automatyzacji w HA i po stronie Tuya	17
6.1	Edytor automatyzacji Home Assistant	17
6.2	Automatyzacje po stronie Tuya — aplikacja Smart Life	17
6.3	Automatyzacje po stronie Tuya — platforma deweloperska (API)	18
6.4	Gdzie trzymać którą automatyzację	19
7	Anatomia automatyzacji	19
7.1	Trzy człony i jeden tryb	19
7.2	Nowa i stara składnia YAML — jak czytać cudze przykłady	20
7.3	Przegląd wyzwalaczy — z krótkimi przykładami	20
7.4	Warunki	21
7.5	Akcje — pełny repertuar	21
7.6	Tryby wykonania	22
8	Szablony Jinja2 — język, w którym HA myśli	22
8.1	Absolutna podstawa: stany są tekstem	22
8.2	Niezbędnik — funkcje i filtry, które załatwiają 95% potrzeb	23
9	Elementy pomocnicze: helpery, skrypty, sceny, blueprinty	23
9.1	Helpery — pamięć i pokręta Twoich reguł	23
9.2	Skrypty i sceny	24
9.3	Blueprinty — automatyzacje z formularzem	24
10	Przepisy — kompletne automatyzacje z komentarzem	24
10.1	Ładowanie w taniej taryfie — wersja dojrzała (helper schedule)	25
10.2	Powiadomienie o końcu ładowania — z danymi sesji	25
10.3	Flagowy: ładowanie nadwyżką z fotowoltaiki z płynną regulacją prądu	26
10.4	Alarm błędu z eskalacją — aż ktoś potwierdzi	27
10.5	Powiadomienie z przyciskami (actionable) — pełny obieg	28
10.6	Watchdog niedostępności — system pilnuje sam siebie	29
10.7	Raport dzienny o 21:00	29

11 Diagnostyka — gdy reguła nie robi tego, co miała	30
11.1 Ślady (traces) — najpotężniejsze i najmniej znane narzędzie	30
11.2 Reszta niezbędnika	30
11.3 Pułapki, które łapią każdego	30
12 Pomiary energii — od encji do panelu Energia	31
12.1 Co encja musi mieć, żeby panel ją przyjął	31
12.2 Taryfy i okresy: utility_meter	31
13 Niezawodność, kopie zapasowe, dostęp zdalny	31
13.1 Kopie zapasowe	31
13.2 Aktualizacje bez dramatów	31
13.3 Dostęp spoza domu — trzy drogi	32
14 Najczęstsze problemy — ściągą	32
15 Słowniczek pojęć i skrótów	33

1. Wstęp — co jest w tym poradniku i jak go czytać

Home Assistant (dalej: HA) potrafi być jednocześnie prostym pilotem do włączania ładowarki i pełnoprawnym systemem sterowania budynkiem, który sam podejmuje decyzje. Różnica między jednym a drugim nie leży w sprzęcie, lecz w zrozumieniu trzech rzeczy: *jak HA jest zbudowany w środku, jak urządzenia się z nim komunikują oraz jak poprawnie zapisywać reguły automatyzacji*. Dokładnie w tej kolejności prowadzi ten poradnik.

Rozdziały 2–5 budują fundament: architektura systemu, protokoły komunikacji (od Wi-Fi i MQTT po Zigbee, Thread/Matter i Modbus) oraz przegląd ekosystemów urządzeń z praktycznymi wskazówkami. Rozdział 6 prowadzi *klik po kliku* przez edytor automatyzacji HA oraz automatyzacje po stronie Tuya (aplikacja i platforma deweloperska), a każdy ekosystem z rozdz. 5 ma teraz instrukcję wdrożenia: najpierw na symulacji (z gotowymi narzędziami w folderze projektu `ha_sim_tools`), potem na rzeczywistym sprzęcie. Rozdziały 7–9 to warsztat automatyzacji: anatomia reguły, język szablonów, elementy pomocnicze. Rozdział 10 zawiera siedem kompletnych, skomentowanych przepisów — od taniej taryfy po dynamiczne ładowanie nadwyżką z fotowoltaiki. Na końcu: diagnostyka (rozdz. 11), pomiary energii (rozdz. 12), niezawodność i dostęp zdalny (rozdz. 13), tabela problemów i słowniczek.

Zasady zapisu są takie same jak w pozostałych dokumentach projektu: ścieżki menu zapisujemy jako **Ustawienia** → **Automatyzacje i sceny**, kod i identyfikatory czcionką maszynową (`sensor.moc`), a każdy skrót rozwijamy przy pierwszym użyciu. Ramki **Dla ciekawych** można pominąć bez szkody dla instrukcji — ale to w nich siedzi zrozumienie, *dlaczego* rzeczy działają.

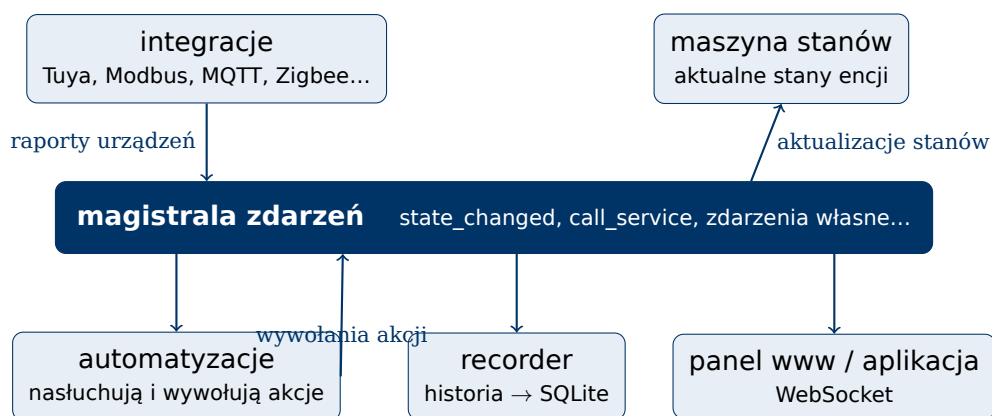
Wszystkie przykłady YAML (YAML Ain't Markup Language — tekstowy format konfiguracji, w którym wcięcia wyznaczają strukturę) zapisano w **nowej składni** automatyzacji, obowiązującej od wydania HA 2024.10; ramka w rozdziale 7 wyjaśnia, jak czytać starszą składnię, wciąż spotykaną w internecie.

2. Jak Home Assistant jest zbudowany

2.1. Encja, stan, zdarzenie — trzy pojęcia, na których stoi wszystko

Encja (entity) to najmniejsza jednostka, jaką HA widzi: jeden czujnik, jeden przełącznik, jedna wartość. Każda encja ma identyfikator złożony z domeny i nazwy (`sensor.ladowarka_moc`, `switch.ladowarka`), bieżący **stan** zapisany zawsze jako tekst (o tym ważnym szczególnie więcej w rozdz. 8) oraz **atrybuty** — dodatkowe pola, np. jednostkę miary czy ikonę. Encje grupują się w **urządzenia** (device — teczka na encje z jednego fizycznego sprzętu), a urządzenia w **obszary** (pokoje, strefy budynku).

Sercem systemu jest **maszyna stanów** (state machine): tabela w pamięci, w której każda encja ma swój aktualny stan. Każda zmiana stanu generuje **zdarzenie** (event) `state_changed`, publikowane na wewnętrznej **magistrali zdarzeń** (event bus). Magistrala działa jak tablica ogłoszeń: kto chce, ten nasłuchuje. Automatyzacje to w istocie właśnie nasłuchiwanie — ich wyzwalacze są subskrypcjami zdarzeń.



Dla ciekawych: Cały HA jest programem w języku Python zbudowanym wokół jednej *pętli zdarzeń* (asyncio): zamiast wielu wątków czekających na urządzenia, jedna pętla obsługuje tysiące operacji naprzemiennie, przełączając się przy każdym oczekiwaniu na sieć. Dlatego HA na Raspberry Pi potrafi obsłużyć setki encji — większość czasu i tak czeka na odpowiedzi urządzeń. Konsekwencja praktyczna: pojedyncza źle napisana integracja, która blokuje pętlę, potrafi „przyciąć” cały system — stąd ostrożność przy dodatkach spoza oficjalnego zestawu.

2.2. Skąd HA pamięta historię: recorder i baza danych

Bieżące stany żyją w pamięci, ale zakładka **Historia** i wykresy wymagają zapisu na dysk. Robi to komponent **recorder**, który strumień zdarzeń zapisuje do bazy danych — domyślnie **SQLite**, czyli bazy w jednym pliku (`home-assistant_v2.db` w katalogu konfiguracji). Obok pełnej historii (domyślnie 10 dni) HA prowadzi **statystyki długoterminowe**: raz na godzinę zapisuje minimum, maksimum i średnią (dla liczników — przyrost) i trzyma je bezterminowo. To dzięki nim panel Energia pokazuje zeszły rok, choć szczegółowa historia dawno się przewinęła.

Wskazówka: Im mniej encji-śmieci, tym mniejsza i szybsza baza. Encje, których nie potrzebujesz, wyłączaj w **Ustawienia** → **Urządzenia i usługi** → **Encje**; głośne, a potrzebne tylko „na teraz” (np. siła sygnału) można wykluczyć z zapisu w konfiguracji recordera.

2.3. Integracje i wpisy konfiguracyjne

Integracja to moduł tłumaczący świat zewnętrzny na encje. Skonfigurowana integracja zostawia po sobie **wpis konfiguracyjny** (config entry) w ukrytym katalogu `.storage` — zapamiętane konto, adres, tokeny. Ta sama integracja może mieć wiele wpisów (dwa konta Tuya, trzy bramki). Z projektu znamy żelazną regułę: *pliki integracji bez wpisu konfiguracyjnego nie robią nic* — kreator (config flow) trzeba przejść do końca.

Dla ciekawych: Obok rejestru encji HA prowadzi rejestry urządzeń i obszarów (również w `.storage`). Dzięki nim zmiana nazwy encji nie psuje automatyzacji budowanych w edytorze graficznym — edytor zapisuje odwołania do identyfikatorów rejestrowych, nie do nazw. W YAML pisanym ręcznie odwołujemy się jednak do `entity_id` — dlatego po zmianie identyfikatora encji trzeba poprawić automatyzacje, które go używają.

2.4. Warianty instalacji — przypomnienie w jednym akapicie

Szczegóły instalacji opisuje przewodnik (dokument towarzyszący); tu tylko mapa: **HA OS** (np. na Raspberry Pi) to pełny zestaw z Supervisorem, sklepem dodatków i kopiami zapasowymi; **HA Container** (nasz wariant Docker na Windows) to sam rdzeń — lżejszy, ale bez sklepu dodatków. W obu przypadkach automatyzacje, integracje i wszystko, co opisuje ten poradnik, działa identycznie. Różnica praktyczna pojawi się tylko tam, gdzie wspominamy o *dodatkach* (add-ons) — w wariantcie kontenerowym ich odpowiedniki uruchamia się jako osobne kontenery Dockera.

3. Jak urządzenia trafiają do Home Assistant

3.1. Odkrywanie: skąd HA wie, że coś jest w sieci

Po starcie HA nasłuchuje kilku mechanizmów ogłaszania się urządzeń i pokazuje znaleziska w **Ustawienia** → **Urządzenia i usługi** jako karty „Wykryto”:

mDNS / zeroconf

urządzenia rozgłaszają w sieci lokalnej swoją nazwę i typ usługi (np. drukarka, ESPHome, Apple HomeKit) — to ten sam mechanizm, dzięki któremu działa adres `homeassistant.local`.

SSDP / UPnP

(Simple Service Discovery Protocol / Universal Plug and Play) — starszy mechanizm ogłoszeń używany m.in. przez telewizory, odtwarzacze i routery.

Nasłuch DHCP

HA obserwuje, jakie urządzenia pobierają adresy z routera; po adresie sprzętowym MAC (Media Access Control — unikalny identyfikator karty sieciowej) i nazwie potrafi rozpoznać znane rodziny sprzętu.

Bluetooth

jeżeli komputer z HA ma adapter Bluetooth (albo tzw. proxy — rozdz. 4.6), HA odbiera rozgłoszenia BLE czujników.

Odkrycie to tylko zaproszenie — integracja i tak przechodzi swój kreator (konto, kod PIN, potwierdzenie).

3.2. Cztery klasy łączności: lokalnie/chmurowo × push/poll

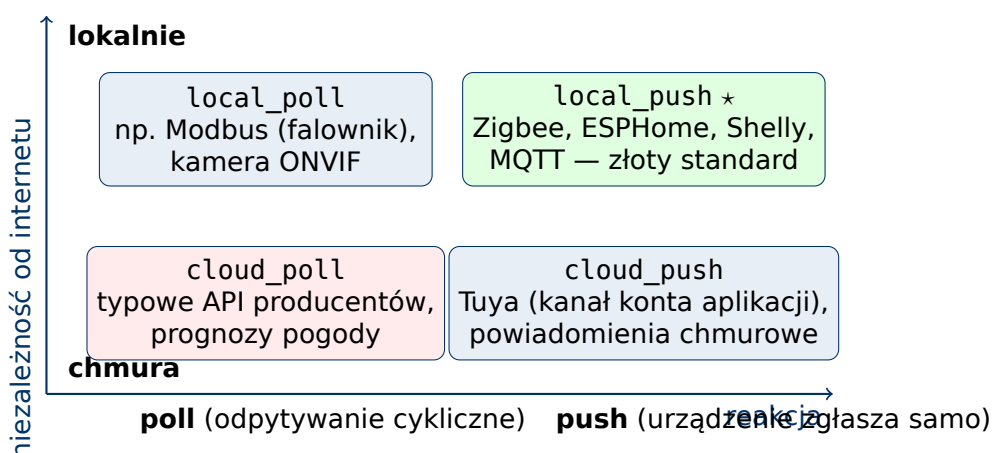
Każda integracja w oficjalnej dokumentacji ma zadeklarowaną tzw. klasę IoT (Internet of Things — internet rzeczy), złożoną z dwóch decyzji:

- **Lokalnie czy przez chmurę?** Lokalnie = HA rozmawia z urządzeniem bezpośrednio po sieci domowej (działa bez internetu). Chmurowo = pośredniczy serwer producenta (jak w Tuya — bez internetu encje stają się „niedostępne”).
- **Push czy poll?** Push = urządzenie samo zgłasza zmiany (natychmiastowe reakcje, mało ruchu). Poll = HA cyklicznie odpytuje (opóźnienie do długości cyklu; typowo 10-60 s).

Kombinacja `local_push` jest złotym standardem (Zigbee, ESPHome, Shelly po włączeniu zdarzeń); `cloud_poll` — najwolniejszym, ale często jedynym, co oferuje producent. Warto znać klasę swoich integracji, bo wyjaśnia ona zachowania: czujnik Zigbee reaguje w ułamku sekundy, a chmurowy odpytywany co minutę — właśnie co minutę.

Dla ciekawych: Skąd chmurowe integracje „push” znają zmiany bez odpytywania? Utrzymują stałe połączenie zwrotne — najczęściej WebSocket (kanał dwukierunkowy otwarty przez przeglądarkowy protokół HTTP) albo MQTT — którym serwer producenta wypycha zdarzenia. Tak działa m.in. kanał konta aplikacji w Tuya: integracja loguje się jak druga aplikacja mobilna i dostaje ten sam strumień powiadomień, który zasila telefon.

3.3. Obie decyzje na jednym rysunku



Gwiazdką oznaczono kwadrant, do którego warto migrować, gdy tylko urządzenie daje taką możliwość.

4. Protokoły — jak naprawdę rozmawia się z urządzeniami

Ten rozdział jest mapą świata urządzeń. Nie trzeba znać wszystkich protokołów, żeby korzystać z HA — ale kilka minut zrozumienia, czym różnią się drogi komunikacji, oszczędza później godziny zgadywania, czemu czujnik „muli”, a przełącznik nie reaguje.

4.1. Wi-Fi i sieć domowa: HTTP, REST, WebSocket

Najprostsza droga: urządzenie jest w tej samej sieci co HA i wystawia **API** (Application Programming Interface — „okienko podawcze” dla programów) po protokole **HTTP** — tym samym, którym przeglądarka pobiera strony. Styl takich API nazywa się **REST** (Representational State Transfer): proste żądania „pobierz stan” (GET) i „ustaw wartość” (POST) pod czytelnymi adresami. HA ma do tego gotowe integracje (`rest`, `rest_command`), a w drugą stronę — **webhooki**: unikalne

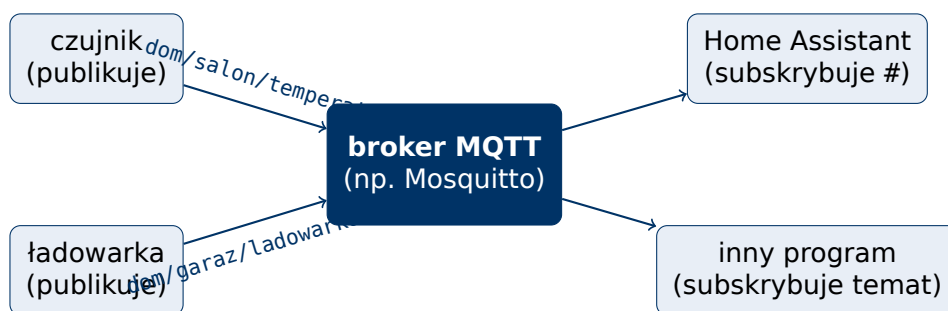
adresy URL, pod które dowolne urządzenie lub usługa może „zapukać” i tym samym wyzwolić automatyzację.

Tam, gdzie potrzebna jest ciągła dwukierunkowa rozmowa (panel www HA, ESPHome, Shelly nowszych generacji), używa się **WebSocket** — połączenia, które po nawiązaniu przez HTTP pozostaje otwarte i pozwala obu stronom wysyłać wiadomości w dowolnym momencie, bez odpytywania.

Dla ciekawych: Cały interfejs HA w przeglądarce to jedna aplikacja rozmawiająca z serwerem właśnie przez WebSocket. Gdy klikasz przełącznik, do HA leci wiadomość `call_service`, a z powrotem — zdarzenie `state_changed`. To dlatego dwa okna przeglądarki aktualizują się nawzajem „same z siebie”.

4.2. MQTT — wspólny język automatyki

MQTT (Message Queuing Telemetry Transport) to lekki protokół „publikuj-subskrybuj”, zaprojektowany dekady temu dla telemetrii rurociągów, a dziś będący lingua franca automatyki domowej. Zamiast rozmawiać każdy z każdym, wszyscy rozmawiają przez pośrednika — **brokera**:



Kluczowe pojęcia, które wystarczą do sprawnego użycia:

Temat (topic)

hierarchiczny adres wiadomości, np. `dom/garaz/ladowarka/stan`. Subskrybować można z wieloznacznikami: `+` (jeden poziom) i `#` (wszystko poniżej).

QoS (Quality of Service)

poziom gwarancji doręczenia: 0 — „wyślij i zapomnij”, 1 — „dotrze przynajmniej raz”, 2 — „dotrze dokładnie raz”. W sieci domowej niemal zawsze wystarcza 0 lub 1.

Retain (wiadomość zatrzymana)

broker pamięta ostatnią wiadomość z tematu i wręcza ją każdemu nowemu subskrybentowi. Dzięki temu po restarcie HA od razu zna stany, zamiast czekać na kolejne raporty.

LWT (Last Will and Testament — „testament”)

wiadomość, którą broker opublikuje za urządzenie, gdy to zniknie bez pożegnania. Tak właśnie encje MQTT wiedzą, że urządzenie jest offline.

Odkrywanie (MQTT Discovery)

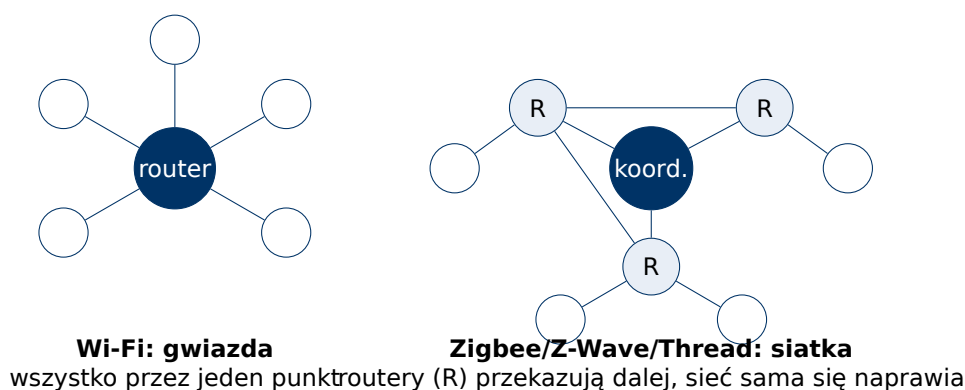
urządzenie publikuje pod specjalnym tematem `homeassistant/.../config` opis swoich encji — a HA tworzy je automatycznie, bez ręcznej konfiguracji. Z tego mechanizmu korzysta m.in. Zigbee2MQTT i Tasmota.

W praktyce: brokera (najczęściej **Mosquitto**) instaluje się jako dodatek HA OS albo osobny kontener, w HA dodaje integrację **MQTT** (adres brokera, login), a urządzenia kieruje na ten sam broker. Domyślny port to 1883 (8883 w wersji szyfrowanej TLS).

4.3. Zigbee — radiowa siatka czujników

Zigbee to radiowy standard krótkiego zasięgu (oparty o normę IEEE 802.15.4) w paśmie 2,4 GHz, zaprojektowany pod *małe zużycie energii*: czujnik drzwi potrafi żyć dwa lata na baterii pastylkowej. Ceną jest przepustowość — to sieć do meldunków „otwarto/zamknięto, 21,5 stopnia”, nie do strumieni wideo.

Sieć Zigbee jest **siatką** (mesh): urządzenia zasilane z gniazdka (żarówki, wtyczki) pełnią rolę **routerów** i przekazują pakiety dalej, a bateryjne **urządzenia końcowe** tylko meldują i śpią. Całością dyryguje **koordynator** — u nas: układ radiowy w formie modułu USB wpiętego do komputera z HA.



W HA są dwie równorzędne drogi obsługi Zigbee:

- **ZHA** (Zigbee Home Automation) — integracja wbudowana: najprostszy start, wszystko w HA;
- **Zigbee2MQTT** — osobny program, który tłumaczy Zigbee na MQTT (rozdz. 4.2); nieco więcej konfiguracji, za to największa na rynku baza obsługiwanych urządzeń i drobiazgowy raporty.

Obu naraz na jednym koordynatorze użyć nie można. Parowanie wygląda tak samo: w integracji włącza się tryb dołączania (permit join), a urządzenie wprowadza w tryb parowania (zwykle przytrzymanie przycisku); po dołączeniu następuje „przesłuchanie” (interview), w którym urządzenie opowiada, co potrafi — z tego powstają encje.

UWAGA: Zigbee i Wi-Fi dzielą pasmo 2,4 GHz. Koordynator wpięty bezpośrednio w port USB komputera bywa zagłuszany przez sam komputer — **zawsze używaj przedłużacza USB** (pół metra wystarczy) i trzymaj koordynator z dala od routera. Przy uporczywych problemach pomaga zmiana kanału Zigbee (15, 20 i 25 leżą między typowymi kanałami Wi-Fi 1/6/11).

4.4. Z-Wave — siatka na osobnej częstotliwości

Z-Wave działa podobnie do Zigbee (siatka, urządzenia bateryjne i routery), ale w Europie nadaje w paśmie **868 MHz**. Dłuższa fala lepiej przenika ściany i — co

równie ważne — nie tłoczy się w zatłoczonym 2,4 GHz. Urządzenia przechodzą obowiązkową certyfikację (mniejsza loteria zgodności niż w Zigbee), za co płaci się wyższą ceną. W HA obsługę zapewnia **Z-Wave JS**: osobny serwer (dodatek albo kontener) plus integracja; kontroler to znów moduł USB. Dołączanie (inclusion) nowszych urządzeń używa zabezpieczeń **S2** z kodem PIN z naklejki.

4.5. Thread i Matter — nowy porządek

Te dwa słowa pojawiają się dziś na pudełkach razem, ale oznaczają co innego — i to rozróżnienie warto mieć w głowie:

- **Thread** to *transport*: radiowa siatka jak Zigbee (ta sama norma IEEE 802.15.4, pasmo 2,4 GHz), tyle że każde urządzenie ma normalny adres **IPv6**. Do połączenia siatki z siecią domową potrzebny jest **border router** (router brzegowy) — tę rolę pełnią m.in. głośniki i przystawki Apple/Google oraz moduły obsługiwane przez HA.
- **Matter** to *język aplikacji*: wspólny standard opisu urządzeń („jestem żarówką, umiem ściemniać”), działający po IP — obojętnie, czy przez Thread, Wi-Fi czy Ethernet. Urządzenie łączy się kodem QR (komisjonowanie), a dzięki **multi-admin** może być jednocześnie w HA i np. w ekosystemie telefonu.

Dla nas najważniejsze: Matter jest **lokalny** (chmura niepotrzebna do działania) i wspierany w HA przez wbudowany serwer Matter. Rynek urządzeń dojrzeewa z każdym rokiem; przy zakupach nowych czujników i wtyczek warto sprawdzać logo Matter/Thread — to inwestycja w niezależność od chmur producentów.

4.6. Bluetooth Low Energy

BLE (Bluetooth Low Energy) to oszczędna odmiana Bluetooth: czujniki (termometry, wagi, opaski) okresowo *rozgłaszają* pomiary, a HA je biernie odbiera — bez parowania. Zasięg pojedynczego adaptera jest mały, ale problem elegancko rozwiązuje **Bluetooth Proxy**: dowolny moduł ESP32 (mikrokontroler za kilkanaście złotych) z oprogramowaniem ESPHome przekazuje odebrane rozgłoszenia do HA przez Wi-Fi. Kilka takich „uszu” rozstawionych po budynku daje pokrycie całości. Otwarty format rozgłoszeń nazywa się **BTHome**.

4.7. Kamery: RTSP i ONVIF

Świat kamer opisują dwa standardy, które znamy już z praktyki (kamera Imou w projekcie): **RTSP** (Real Time Streaming Protocol) to sposób pobierania samego strumienia wideo (adres zaczyna się od `rtsp://`, typowy port 554, kodeki H.264/H.265), a **ONVIF** — warstwa zarządzania: odkrywanie kamery, lista strumieni, sterowanie głowicą PTZ (pan-tilt-zoom), zdarzenia detekcji ruchu. W HA: integracja **ONVIF** tworzy encje kamery i zdarzeń, a podgląd w panelu serwuje wbudowany **go2rtc**, który potrafi przetłumaczyć strumień na niskolatencyjny WebRTC do przeglądarki.

Dla ciekawych: Nagrywanie i rozpoznawanie obiektów (osoba/samochód/zwierzę) to domena zewnętrznego programu **Frigate**, który konsumuje strumienie RTSP,

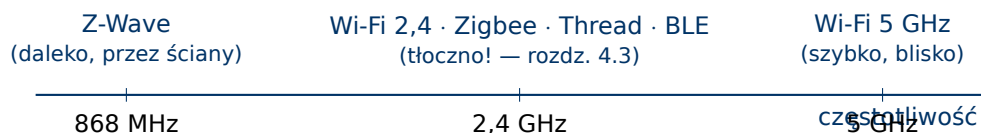
a wyniki publikuje przez MQTT — kolejny przykład, jak MQTT skleja niezależne systemy w całość.

4.8. Modbus — język przemysłu (i falowników)

Modbus pochodzi z automatyki przemysłowej lat 70. i przeżył wszystkich swoich następców dzięki prostocie: urządzenie wystawia ponumerowane **rejestry** (komórki 16-bitowe), które można czytać i zapisywać. Odmiana **RTU** biegnie po skrętce RS-485, odmiana **TCP** — po zwykłej sieci (port 502). Wartości większe niż 16 bitów składa się z dwóch rejestrów, a liczby ułamkowe przekazuje jako całkowite ze skalą (np. 2305 = 230,5 V) — dokładnie ten mechanizm znamy z symulatora falownika SUN2000 w projekcie. W HA: dedykowane integracje (np. huawei_solar) albo uniwersalna integracja modbus, w której rejestry mapuje się ręcznie w YAML.

4.9. Ściąga porównawcza

Technologia	Pasmo	Topologia	Zasilanie urządzeń	Typowe zastosowanie
Wi-Fi	2,4 / 5 GHz	gwiazda	sieciowe	ładowarki, kamery, gniazdka
Zigbee	2,4 GHz	siatka	baterie / sieciowe	czujniki, żarówki, piloty
Z-Wave	868 MHz (EU)	siatka	baterie / sieciowe	zamki, rolety, czujniki
Thread / Matter	2,4 GHz / IP	siatka / dowolna	baterie / sieciowe	nowe czujniki i wtyczki
BLE	2,4 GHz	rozgłoszenia	baterie	termometry, wagi, opaski
MQTT	(po sieci IP)	przez brokera	—	spoiwo systemów
Modbus	RS-485 / IP	żądanie-odpowieź	sieciowe	falowniki, liczniki, HVAC



5. Ekosystemy w praktyce — czym i jak łączyć konkretne urządzenia

5.1. Urządzenia Tuya (w tym ładowarki AMPERE POINT)

Pełną, przeciwiczoną w boju procedurę opisują dokumenty towarzyszące (przewodnik instalacji oraz raport xtend_tuya); tu tylko esencja architektoniczna. Urządzenie Tuya rozmawia **wyłącznie** z chmurą producenta (klasa cloud_push przez

kanal konta aplikacji + cloud_poll przez OpenAPI). Wszystkie wielkości płyną jako **punkty danych** (DP, data point): ponumerowane zmienne o zadeklarowanym typie i skali. Oficjalna integracja pokazuje ich okrojony, „ugrzeczniiony” podzbiór; dodatek **xtend_tuya** dokłada resztę, a tryb **DP Instruction** na platformie deweloperskiej odślania komplet. Alternatywą niezależniącą od chmury jest **tuya-local** (rozmowa po sieci lokalnej z użyciem klucza urządzenia) — opłacalna tam, gdzie liczy się czas reakcji i odporność na przerwy internetu.

5.1.1. Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt

Symulacja A — udawana ładowarka w HA (plik ha_sim_tools/symulator_ladowarki.yaml w folderze projektu). Pakiet tworzy komplet encji zachowujących się jak ładowarka: status charger_free/insert/charging, moc wyliczaną z limitu prądu ($I \times 3 \times 230 \text{ V}$), energię narastającą i dzienną, nastawę prądu 6-16 A i przełącznik, który — jak w rzeczywistości — nie wystartuje bez „włożonej wtyczki”.

1. Otwórz C:\HomeAssistant\config\configuration.yaml (Notatnik wystarczy) i dopisz na końcu (raz, jeśli sekcji jeszcze nie ma):

```
homeassistant:
  packages: !include_dir_named packages
```

2. W config utwórz folder packages i skopiuj do niego plik symulator_ladowarki.yaml.
3. Zrestartuj kontener homeassistant. W **Ustawienia** → **Urządzenia i usługi** → **Encje** wyszukaj „symulowana” — komplet na miejscu.
4. Sterowanie symulacją: włącz input_boolean.sym_wtyczka („auto podjechało”), potem przełącznik switch.symulowana_ladowarka — moc skoczy, energia zacznie rosnąć. Zmieniaj number.symulowana_ladowarka_limit_pradu i patrz, jak moc podąża.
5. Testuj przepisy z rozdz. 10 podmieniając wyłącznie identyfikatory wg tabeli poniżej; nadwyżkę PV dostarcza drugi symulator projektu (pv_sim — falownik SUN2000 z licznikiem eksportu).

Encja symulacyjna	Odpowiednik na sprzęcie (xtend_tuya)
sensor.symulowana_ladowarka_status	sensor.*_work_state (Q37: x_work_state)
sensor.symulowana_ladowarka_moc	sensor.*_power_total / moc z pakietu fazy
sensor.symulowana_ladowarka_energia	sensor.*_forward_energy_total
number.symulowana_ladowarka-_limit_pradu	number.*_charge_cur_set (Q37: x_charge_current)
switch.symulowana_ladowarka	switch.* (Q37: switch_led)

Symulacja B — wirtualne urządzenie na platformie Tuya (nauka punktów danych i API, bez sprzętu): **Devices** → **Add Device** → **Add Virtual Devices for Debugging** → wybierz kategorię produktu → urządzenie dostaje własny device_id. Sterowanie i podgląd: **Cloud** → **API Explorer** → **Device Control** → **Query Properties / Send Commands** (wklej device_id).

UWAGA: Urządzenie wirtualne istnieje tylko po stronie projektu API — nie należy do konta aplikacji Smart Life, więc **nie pojawi się w HA** przez `xtend_tuya`. Służy do nauki DP i testów API, nie do testów automatyzacji HA — od tego jest Symulacja A.

Na sprzęcie: pełna procedura podłączenia ładowarek to etapy 3–7 przewodnika instalacji (dokument towarzyszący). Po jej wykonaniu wdrożenie automatyzacji sprowadza się do podmiany identyfikatorów wg tabeli powyżej — logika przepisów pozostaje bez zmian. Dobra praktyka projektu: każdą regułę najpierw uruchom na Symulacji A, wymuś wszystkie gałęzie (start, korekta, stop, błąd), obejrzyj Ślady — i dopiero wtedy przełącz na encje rzeczywiste.

5.2. ESPHome — własne urządzenia w godzinę

ESPHome zasługuje na osobną gwiazdkę: to projekt, w którym *opisujesz* urządzenie w YAML, a system *kompiluje* z tego gotowe oprogramowanie dla mikrokontrolera **ESP32/ESP8266**. Czujnik temperatury do rozdzielnic, licznik impulsów wodomierza, przekaźnik — wszystko sprowadza się do kilkunastu linii:

```
esphome:
  name: czujnik-rozdzielnica

esp32:
  board: esp32dev

wifi:
  ssid: "NazwaSieci"
  password: "HasloSieci"

api:          # szyfrowane połączenie z HA (local_push)
ota:          # aktualizacje przez Wi-Fi

sensor:
  - platform: dallas_temp
    pin: GPIO4
    name: "Temperatura rozdzielnic"
    update_interval: 30s
```

Po pierwszym wgraniu (kablem USB) kolejne aktualizacje lecą przez Wi-Fi (**OTA** — Over The Air), a HA wykrywa urządzenie sam (mDNS) i łączy się z nim natywnym, szyfrowanym API w klasie `local_push`. To najkrótsza droga od pomysłu „przysłałby się czujnik X” do encji w HA — i fundament wspomnianych Bluetooth Proxy.

5.2.1. Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt

Symulacja — ESPHome bez żadnego układu (platforma `host`: program kompiluje się i działa na komputerze, a HA widzi go jak fizyczne urządzenie). Na Windows uruchamiamy go w WSL (mamy je od instalacji Dockera). Plik gotowy: `ha_sim_tools/esphome_host_sim.yaml` — symulowana temperatura (sinusoida z szumem) i przekaźnik.

1. Terminal → `wsl` → w środku: `pip install esphome`.
2. Skopiuj plik narzędzi do WSL i uruchom:

```
cp /mnt/c/Users/Lenovo/Desktop/AMPERE_POINT/ha_sim_tools/esphome_host_sim.yaml ~/
esphome run ~/esphome_host_sim.yaml
```

Po kompilacji log pokaże nasłuch API na porcie 6053. Zostaw okno otwarte.

3. W HA: **Dodaj integrację** → **ESPHome** → Host: `host.docker.internal`, Port: 6053 → encje temperatury i przekaźnika gotowe do automatyzacji.

Ograniczenie trybu host: brak dostępu do fizycznych nóżek i magistral (GPIO, I2C) — symuluje się logikę i integrację, nie elektronikę.

Na sprzęcie (moduł ESP32 DevKit, koszt rzędu 30–40 zł):

1. Podłącz płytkę kablem USB (danych, nie samego zasilania); Windows zwykle sam instaluje sterownik przetwornika (CP210x/CH340) — jeśli port COM się nie pojawi, doinstaluj sterownik producenta.
2. W pliku YAML zamień sekcję `host:` na `esp32:` `board: esp32dev` i dodaj sekcję `wifi:` (przykład w podrozdz. 5.2).
3. `esphome run czujnik.yaml` → wybierz z listy port COM → pierwsze wgranie kablem; każde kolejne — przez OTA (radiowo), płytka może już wisieć w rozdzielnic.
4. HA wykryje urządzenie przez mDNS (karta „Wykryto”); klucz szyfrowania API znajdziesz/ustawisz w YAML.

5.3. Shelly i podobne: Wi-Fi, ale lokalnie

Przekaźniki i mierniki **Shelly** pokazują, że Wi-Fi nie musi oznaczać chmury: mają lokalne API (nowsze generacje: WebSocket/RPC), integrację w HA klasy `local_push` i opcjonalnie MQTT. Montowane w puszkach za istniejącymi włącznikami dodają „inteligencję” bez wymiany osprzętu — a pomiar mocy w wersjach PM czyni z nich tanie liczniki częściowe do panelu Energia.

5.3.1. Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt

Symulacja — udawane urządzenie przez MQTT (uczy brokera, tematów, discovery i retain — pełne komendy w pliku `ha_sim_tools/mqtt_fake_device.md`):

1. Postaw brokera Mosquitto (kontener; plik konfiguracyjny z dwiema liniami `listener 1883 + allow_anonymous true` — komenda `docker run` w pliku narzędzi).
2. W HA: **Dodaj integrację** → **MQTT** → broker `host.docker.internal`, port 1883.
3. Opublikuj (z retain) dwa ogłoszenia konfiguracyjne na tematach `homeassistant/switch/.../config` i `homeassistant/sensor/.../config` — w HA *samo* pojawi się urządzenie „Symulowane gniazdko MQTT” z przełącznikiem i mocą. To jest MQTT Discovery w akcji.
4. Publikuj stany (ON/liczby watów) i patrz na encje; w drugim oknie `mosquitto_sub` na temacie `.../set` pokazuje na żywo komendy, które HA wysyła, gdy klikasz przełącznik. Kwadrans tej zabawy uczy MQTT lepiej niż godzina lektury.

Na sprzęcie (przekaźnik/miernik Shelly):

1. Montaż w puszcze = praca przy 230 V — wykonuje osoba z uprawnieniami. Wersje wtyczkowe (Plug) wpinasz po prostu do gniazdka.
2. Pierwsze uruchomienie: Shelly rozgłasza własną sieć Wi-Fi shelly...; połącz się telefonem → przeglądarka → 192.168.33.1 → zakładka Wi-Fi → podaj dane **domowej sieci 2,4 GHz** → zapisz (urządzenie zrestartuje się już w Twojej sieci).
3. HA wykryje je samo (karta „Wykryto” — mDNS) → **Konfiguruj** → gotowe: encje przełącznika i pomiarów, klasa local_push.
4. Opcjonalnie, w duchu lokalności: w panelu www Shelly wyłącz chmurę producenta — HA jej nie potrzebuje.

5.4. Telefon jako urządzenie: aplikacja Companion

Aplikacja **Home Assistant Companion** (Android/iOS) robi z telefonu pełnoprawne urządzenie: dostarcza device_tracker (obecność w strefach — rozdz. 10), dziesiątki sensorów (bateria, ładowanie, sieć Wi-Fi, kroki), a przede wszystkim usługę **notify** — kanał powiadomień, którego używają nasze przepisy. Powiadomienia mogą być *aktywne* (actionable): z przyciskami, których naciśnięcie wraca do HA jako zdarzenie — przepis 10.5 pokazuje pełny obieg.

5.4.1. Wdrożenie krok po kroku — najpierw symulacja, potem sprzęt

Symulacja — testy bez telefonu (przydatne też później, przy debugowaniu):

1. Treści powiadomień: w automatyzacjach podstaw tymczasowo usługę notify.persistent_notification — komunikaty lądują jako dzwoneczek w panelu HA, z pełnym renderowaniem szablonów (energia, koszt). Zero telefonu, natychmiastowy podgląd.
2. Przyciski (actionable): **Narzędzia deweloperskie** → **Zdarzenia** → w polu „Typ zdarzenia” wpisz mobile_app_notification_action, w danych:

```
action: START_LADOWANIA
```

→ **Wyślij zdarzenie**. Automatyzacja-odbiornik z przepisu 10.5 zadziała identycznie, jakby ktoś nacisnął przycisk na telefonie — sprawdź jej Ślad.

Na sprzęcie (telefon domownika):

1. Zainstaluj **Home Assistant** (Google Play / App Store) → przy pierwszym uruchomieniu podaj adres serwera: w sieci domowej http://IP-komputera:8123 (aplikacja zwykle znajdzie go sama przez mDNS).
2. Zaloguj się kontem HA tej osoby (osobne konta = osobne śledzenie stref i osobne kanały powiadomień).
3. Zezwolenia: **powiadomienia** — zawsze; **lokalizacja „zawsze”** — tylko jeśli chcesz automatyzacji obecności (strefy).
4. Nazwę urządzenia sprawdzisz w aplikacji: **Ustawienia** → **Aplikacja towarzysząca** — od niej pochodzi usługa notify.mobile_app_<nazwa>; pełną listę usług zobaczysz w **Narzędzia deweloperskie** → **Akcje** wpisując „notify”.

5. Strefy rysujesz w **Ustawienia** → **Obszary, etykiety i strefy** → **Strefy** (dom masz z instalacji; dodaj np. „Warsztat” z promieniem 100 m) — encja osoby przyjmuje wtedy stany home/Warsztat/not_home, gotowe wyzwalacze zone.

5.5. Co wybierać na przyszłość — krótki Kompas

Do istniejącej instalacji AMPERE POINT (ładowarki Tuya + planowany falownik po Modbus) najbardziej naturalne rozszerzenia to: czujniki i wtyczki **Zigbee** (tania, dojrzała siatka — wymaga tylko koordynatora USB) lub **Matter/Thread** (nowocześniejsza alternatywa), **ESPHome** do pomiarów nietypowych oraz **Shelly** tam, gdzie trzeba wpiąć się w istniejącą elektrykę. Wspólny mianownik: wszystko działa lokalnie, więc awaria internetu zatrzymuje tylko chmurę Tuya — nie dom.

5.5.1. Ścieżki wdrożenia dla kierunków z kompasu

Zigbee — uczciwie: symulacji nie ma (to fizyczne radio; potrzebny koordynator za ok. 60–80 zł). Na sprzęcie:

1. Kup koordynator USB (sprawdzony standard: układy Silicon Labs EFR32 lub TI CC2652) i **przedłużacz USB** (rozd. 4.3 — zakłócenia).
2. **UWAGA:** Nasz wariant instalacji (Docker Desktop na Windows) **nie przekazuje urządzeń USB do kontenerów**. Realne opcje: (a) przekierowanie przez usbipd-win — do zrobienia, ale to najbardziej techniczny krok tego poradnika; (b) zalecane: koordynator wpinamy do Raspberry Pi z wariantu R przewodnika — HAOS widzi USB wprost i podpowiada integrację sam.
3. HA: **Dodaj integrację** → **Zigbee Home Automation (ZHA)** → wybierz port urządzenia → kreator utworzy sieć.
4. Parowanie: wpis ZHA → **Dodaj urządzenie** (sieć otwiera się na 60 s) → na czujniku przytrzymaj przycisk parowania ok. 5 s (dioda mruga) → po „przesłuchaniu” pojawiają się encje. Urządzenia zasilane sieciowo dodawaj najpierw — rozbudowują siatkę.

Matter/Thread: w HAOS serwer Matter jest wbudowany — dodanie urządzenia: aplikacja Companion → **Dodaj urządzenie Matter** → skan kodu QR z pudełka; dla urządzeń Thread potrzebny w domu border router. W wariantcie kontenerowym Matter wymaga osobnego kontenera matter-server — kolejny argument za docelową przesiadką na wariant R.

Falownik po Modbus — tu symulację mamy klasy premium: pv_sim z folderu projektu (pełne README w pv_sim/README_URUCHOMIENIE.md). Skrót klik po kliku: python sun2000_sim.py → HACS: pobierz **Huawei Solar** → restart → **Dodaj integrację** → **Huawei Solar** → Modbus TCP, host host.docker.internal, port 502 → komplet encji falownika i licznika (w tym sensor mocy eksportu do przepisu PV). **Na sprzęcie zmienia się dokładnie jedno pole:** host = adres IP falownika/SDongle w sieci lokalnej (Modbus TCP musi być włączony w konfiguracji instalatorskiej falownika). Cała reszta — integracja, encje, automatyzacje — zostaje bez zmian; na tym polegał sens budowy symulatora.

6. Klik po kliku: wdrażanie automatyzacji w HA i po stronie Tuya

Ten rozdział to czysta instrukcja obsługi narzędzi — każde kliknięcie po kolei. Teorię klocków (wyzwalacze, warunki, akcje) wyjaśnia rozdział 7; tu uczymy się *gdzie* się je klika.

6.1. Edytor automatyzacji Home Assistant

1. **Ustawienia** → **Automatyzacje i sceny** → niebieski przycisk + **Utwórz automatyzację** (prawy dolny róg) → **Utwórz nową automatyzację** (pozycje z listy blueprintów na razie pomijamy).
2. Sekcja **Kiedy** (wyzwalacze): + **Dodaj wyzwalacz** → z listy wybierz rodzaj, np. **Encja** → **Stan liczbowy** → w polu *Encja* zacznij pisać nazwę (podpowiadanie działa po 2-3 znakach) → wypełnij *Powyżej/Poniżej* i opcjonalnie *Przez* (czas utrzymania). Kolejne wyzwalacze dodajesz tym samym przyciskiem — pamiętaj: łączą się przez LUB.
3. Sekcja **I jeżeli** (warunki, opcjonalne): + **Dodaj warunek** → np. **Czas i lokalizacja** → **Czas** → zakres godzin lub dni tygodnia.
4. Sekcja **Wtedy wykonaj** (akcje): + **Dodaj akcję**. Trzy najczęstsze drogi: **Urządzenie** (najprościej: wybierz urządzenie i co ma zrobić), **Wykonaj akcję** (pełna lista usług systemu — np. `number.set_value`; tu trafisz też do powiadomień notify), oraz **Elementy konstrukcyjne** (Wybierz/Jeżeli-to/Czekaj/Powtarzaj — struktury z rozdz. 7).
5. Prawy górny róg → ikona **dyskiety (Zapisz)** → nadaj **nazwę** (opis warto uzupełnić — przyszły Ty podziękuje) → **Zapisz**.
6. Po zapisaniu, na liście automatyzacji, każda pozycja ma: **suwak** (włącz/wyłącz regułę bez kasowania) oraz menu „...” z pozycjami **Uruchom** (wykonuje *tylko sekcję akcji* — pomija wyzwalacze i warunki; szybki test skutków), **Ślady** (zapis przebiegów — rozdz. 11), **Edytuj w YAML**, **Duplikuj**, **Usuń**.

Wskazówka: W edytorze każdy pojedynczy klocek (wyzwalacz, warunek, akcja) ma własne menu „...” → **Edytuj w YAML** — można więc 90% reguły wyklikać, a tylko jeden warunek dopisać szablonem. Dokładnie tak najszybciej przenosi się przepisy z rozdz. 10: cała reguła przez *Edytuj w YAML* na poziomie automatyzacji, potem kosmetyka w edytorze wizualnym.

6.2. Automatyzacje po stronie Tuya — aplikacja Smart Life

Automatyzacje utworzone w aplikacji wykonują się **w chmurze Tuya** (a proste reguły jednego urządzenia — w samym urządzeniu), więc działają nawet przy wyłączonym komputerze z HA. To ich największa zaleta i powód, by trzymać tam zabezpieczenia „ostatniej szansy”.

1. Dolny pasek aplikacji → zakładka **Scena** → + (prawy górny róg).
2. Wybierz rodzaj warunku „**Gdy**”: *Gdy zmieni się status urządzenia* (najczęstsze), *Harmonogram* (o godzinie), *Gdy zmieni się pogoda*.

3. Dla warunku urządzeniowego: wybierz **ładowarkę** z listy → wybierz **funkcję** (punkt danych, np. stan pracy / moc) → wybierz wartość progową lub stan.
4. Sekcja „**Wtedy**”: **Uruchom urządzenie** → wybierz urządzenie → funkcja (np. Przełącznik: Wyl.) — albo **Wyślij powiadomienie**.
5. Opcjonalnie: *Okres ważności* (np. reguła aktywna tylko nocą) → **Zapisz** → automatyzacja pojawia się na liście z suwakiem włącz/wyłącz.

Ograniczenia wobec HA: tylko proste „jeżeli-to” (bez histerezy, szablonów, zmiennych), tylko urządzenia Tuya, logika w chmurze producenta. Reguła kciuka z rozdz. 6.4: bezpieczeństwo — tu; inteligencja — w HA.

6.3. Automatyzacje po stronie Tuya — platforma deweloperska (API)

Na platform.tuya.com automatyzacje chmurowe (*linkage rules*) tworzy się przez **API** — platforma nie ma klikanego kreatora reguł, ma za to **API Explorer**: formularz, w którym wywołania API klika się bez pisania programu. Usługę *Smart Home Scene Linkage* autoryzowaliśmy w projekcie przy jego zakładaniu (oznaczona jako przestarzała, lecz działająca; następcy adresują scenariusze przemysłowe).

1. Zbierz identyfikatory: **Devices** → **Link App Account** — skopiuj UID konta; następnie **Cloud** → **API Explorer** → usługa *Smart Home Basic Service* → *Get the home list* → wklej uid → **Submit Request** → z odpowiedzi zanotuj *home_id*.
2. API Explorer → usługa *Smart Home Scene Linkage* → *Add Automation* (POST /v1.0/homes/{home_id}/automations) → w polu *Request Body* wklej regułę, np. „gdy moc ładowarki przekroczy próg, wyłącz przełącznik”:

```
{
  "name": "Limit mocy - wylacz",
  "conditions": [{
    "entity_id": "ID_URZADZENIA",
    "entity_type": 1,
    "expr": { "comparator": ">", "status_code": "power_total",
              "status_value": 20000 }
  }],
  "actions": [{
    "entity_id": "ID_URZADZENIA",
    "action_executor": "dpIssue",
    "executor_property": { "switch": false }
  }],
  "match_type": 1
}
```

3. **Submit Request** — odpowiedź zwraca *automation_id*. Regułę włączając/wyłączając endpointy *Enable/Disable automation*, a listę pokazuje *Get automation list*.

Szczerza ocena: to droga dla integracji programistycznych (systemy nadrzędne, aplikacje własne). W codziennej praktyce projektu automatyzacje chmurowe wygodniej klikać w Smart Life (podrozdz. 6.2), a złożoną logikę trzymać w HA. API Explorer pozostaje bezcenny do *podglądania* urządzeń (Device Control → Query Properties) — to nasze okno na surowe punkty danych.

6.4. Gdzie trzymać którą automatyzację

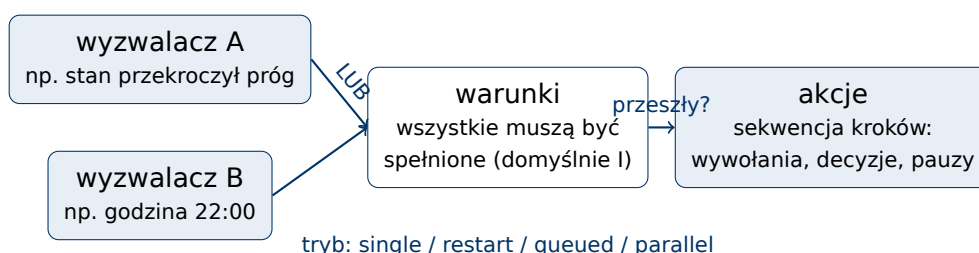
Reguła	Najlepsze miejsce i powód
Zabezpieczenie „ostatniej szansy” (wyłącz przy przegrzaniu, limit twardy)	urządzenie / Smart Life — działa bez HA, bez komputera, bez sieci lokalnej
Komfort i logika wielourządzeniowa (taryfy, nadwyżka PV, powiadomienia z danymi)	HA — histereza, szablony, urządzenia wielu marek w jednej regule
Reguły wymagane przez system nadrzędny (rozliczenia, aplikacja firmowa)	API platformy (linkage rules) — zarządzane programowo, per konto

UWAGA: Jedna reguła — jedno miejsce. Ta sama logika wdrożona równolegle w HA i w Smart Life to przepis na „duchy”: ładowarka włącza się „sama”, a ślad w HA milczy, bo sprawcą jest chmura. Przy diagnozie zawsze sprawdź listę scen w aplikacji.

7. Anatomia automatyzacji

7.1. Trzy człony i jeden tryb

Każda automatyzacja składa się z tych samych klocków: **wyzwalaczy** (triggers — „kiedy w ogóle się obudzić?”), opcjonalnych **warunków** (conditions — „czy na pewno działać?”) i **akcji** (actions — „co zrobić?”). Nad całością czuwa **tryb** (mode), który rozstrzyga, co się stanie, gdy reguła zostanie wyzwolona ponownie, zanim skończyła poprzedni przebieg.



Dwie zasady, które oszczędzają godziny zdiwienia:

- **Wyzwalacze łączą się przez LUB** — zadziałanie któregośkolwiek budzi regułę. **Warunki łączą się przez I** — wszystkie muszą przepuścić (chyba że jawnie użyjesz or).
- Wyzwalacze reagują na *zmianę* (przekroczenie progu, wejście w stan), a nie na *trwanie*. Jeżeli moc jest powyżej progu „od zawsze”, wyzwalacz progowy nie strzeli, dopóki wartość nie spadnie i nie przekroczy progu ponownie. Do pilnowania trwania służą warunki i parametr for.

7.2. Nowa i stara składnia YAML — jak czytać cudze przykłady

Od wydania 2024.10 edytor zapisuje automatyzacje kluczami w liczbie mnogiej: `triggers:`, `conditions:`, `actions:`; typ wyzwalacza wskazuje klucz `trigger:`, a wywołanie usługi — `action:`. Starsza składnia (`trigger:` z `platform:` oraz `action:` z `service:`) **działa nadal i bezterminowo** — oba zapisy można nawet mieszać. W internecie spotkasz oba; tłumaczenie jest mechaniczne:

Stary zapis	Nowy zapis
<code>trigger: (lista)</code>	<code>triggers:</code>
<code>- platform: state</code>	<code>- trigger: state</code>
<code>condition: (lista)</code>	<code>conditions:</code>
<code>action: (lista)</code>	<code>actions:</code>
<code>- service: switch.turn_on</code>	<code>- action: switch.turn_on</code>

7.3. Przegląd wyzwalaczy — z krótkimi przykładami

state — zmiana stanu

najczęstszy. Opcje `from/to` zawężają przejście, a `for` żąda, by nowy stan *utrzymał się* przez zadany czas (licznik zeruje się przy każdej zmianie!):

```
triggers:
- trigger: state
  entity_id: sensor.ladowarka_status
  to: "charger_charging"
  for: "00:00:10"
```

numeric_state — próg liczbowy

strzela w momencie *przekroczenia* progu (`above/below`), również z `for`:

```
triggers:
- trigger: numeric_state
  entity_id: sensor.moc_eksportu
  above: 3000
  for: "00:05:00"
```

time i time_pattern

konkretna godzina (`at: "22:00:00"`, też encja `input_datetime`) albo rytm (`time_pattern` z `minutes: "/5"` = co pięć minut). Rytmów używaj oszczędnie — to odpytywanie świata na siłę; zwykle lepszy jest wyzwalacz zdarzeniowy.

sun — słońce

wschód/zachód z przesunięciem: `event: sunset, offset: "-00:30:00"`. Do progów jasności lepsza bywa elewacja słońca w warunku szablonowym.

template — dowolny warunek jako wyzwalacz

strzela, gdy szablon (rozdz. 8) zmieni wynik na prawdę; z `for` pilnuje trwania. Najpotężniejszy i najłatwiejszy do przekombinowania.

zone — strefy

wejście/wyjście osoby (śledzonej przez aplikację Companion) ze strefy zdefiniowanej na mapie: `zone: zone.dom, event: enter`.

webhook

unikalny adres URL; żądanie HTTP pod ten adres wyzwala regułę — najprostszy most z zewnętrznymi systemami.

event, mqtt, device

surowe zdarzenie z magistrali (np. akcja z powiadomienia — przepis 10.5), wiadomość MQTT na temacie, albo „wyzwalacz urządzenia” generowany przez edytor. Ten ostatni jest wygodny w kreatorze, ale w ręcznym YAML czytelniejsze są wyzwalacze encji.

homeassistant — start systemu

event: start pozwala odtworzyć pożądane stany po restarcie (patrz pułapki, rozdz. 11).

Każdemu wyzwalaczowi można nadać id — etykietę, po której akcje rozpoznają, co obudziło regułę (używa jej przepis taryfowy w rozdz. 10).

7.4. Warunki

Warunki mają te same rodzaje co wyzwalacze (state, numeric_state, time, sun, zone, template) plus spójniki and/or/not. Dwa niuanse:

- time jako warunek przyjmuje też dni tygodnia: weekday: [mon, tue, wed, thu, fri].
- Warunek state z for pyta „czy encja jest w tym stanie *co najmniej od...*” — to uzupełnienie wyzwalacza, nie duplikat.

7.5. Akcje — pełny repertuar

Akcje wykonują się po kolei, jak skrypt. Poza zwykłymi wywołaniami (action: switch.turn_on z target:) masz do dyspozycji strukturę sterującą:

choose / default

rozgałęzienie „pierwszy pasujący wariant”: lista opcji, każda z własnymi warunkami i sekwencją. Idealne w parze z trigger_id.

if / then / else

prostsza, dwugłęziowa wersja choose.

repeat

pętla: count (n razy), while (dopóki warunek), until (aż warunek). Wewnątrz dostępny licznik repeat.index.

wait_for_trigger / wait_template

zatrzymaj się i czekaj na zdarzenie albo na spełnienie szablonu; zawsze dodawaj timeout i decyduj o continue_on_timeout, inaczej reguła może wisieć godzinami.

delay

pauza. Pamiętaj: restart HA przerywa trwające delay i wait — automatyzacja „zapomina”, że czekała. Długie odliczania bezpieczniej realizować pomocnikiem timer, który potrafi wznowić się po restarcie.

parallel

gałęzie równoległe (np. powiadom trzy osoby naraz).

stop

zakończ przebieg (z opcjonalnym komunikatem w śladzie); wariant z error: true oznacza przebieg jako błędny.

variables

zmienne lokalne przebiegu — policz raz, użyj wielokrotnie (przepis PV, rozdz. 10.3).

7.6. Tryby wykonania

Tryb	Zachowanie przy ponownym wyzwoleniu w trakcie pracy
single (domyślny)	nowy przebieg jest <i>pomijany</i> (w dzienniku: ostrzeżenie). Dobry do reguł, które nie powinny się nakładać.
restart	bieżący przebieg jest przerywany, rusza nowy. Naturalny dla „światło na 5 minut od ostatniego ruchu”.
queued	przebiegi ustawiają się w kolejkę (limit max). Dla sekwencji, które muszą wykonać się wszystkie, po kolei.
parallel	przebiegi biegną równolegle (limit max). Dla reguł obsługujących wiele niezależnych obiektów naraz.

UWAGA: Najczęstsza zagadka „czemu automatyzacja nie zadziałała” u początkujących to tryb single + długie delay/wait w akcjach: reguła wciąż „pracuje”, więc kolejne wyzwolenia przepadają. Rozwiązania: tryb restart, krótsze czekania albo przeniesienie odliczania do pomocnika timer.

8. Szablony Jinja2 — język, w którym HA myśli

Jinja2 to język szablonów: tekst, w którym fragmenty w podwójnych klamrach `{{ ... }}` są obliczane, a bloki `{% ... %}` sterują logiką. Szablony pojawiają się wszędzie: w wyzwalaczach template, warunkach, treściach powiadomień, encjach szablonowych. Poligonem doświadczalnym jest **Narzędzia deweloperskie** → **Szablon** — wynik widać na żywo, w miarę pisanía.

8.1. Absolutna podstawa: stany są tekstem

Funkcja `states('sensor.moc')` zwraca stan encji — **zawsze jako tekst**, nawet gdy wygląda jak liczba. Porównanie tekstu z liczbą to klasyczna pułapka: `'7' > 30` nie znaczy tego, co podpowiada intuicja. Dlatego liczby zawsze konwertujemy filtrem, z wartością awaryjną na wypadek stanu nieliczbowego:

```
{{ states('sensor.moc_eksportu') | float(0) }}
{{ (states('sensor.energia') | float(0)) > 3.5 }}
```

Wartość awaryjna (tu: 0) ratuje przed stanami unknown (jeszcze nieznany) i unavailable (integracja straciła urządzenie) — oba zdarzają się każdemu czujnikowi, choćby po restarcie. Czystszy test dostępności: `has_value('sensor.moc')`.

8.2. Niezbędnik — funkcje i filtry, które załatwiają 95% potrzeb

Wyrażenie	Co robi
<code>states('sensor.x')</code>	stan encji jako tekst
<code>is_state('switch.x', 'on')</code>	czy encja jest w danym stanie (bezpieczne porównanie)
<code>state_attr(enc, 'atrybut')</code>	wartość atrybutu encji (albo None), np. ('sun.sun', 'elevation')
<code>has_value('sensor.x')</code> <code>float(0)</code> , <code>int(0)</code> <code>round(1)</code> <code>default('---')</code>	czy stan jest liczbowo/logicznie użyteczny konwersja na liczbę z wartością awaryjną zaokrąglenie wartość zastępcza, gdy zmienna nie istnieje
<code>now()</code>	bieżący czas (obiekt datetime, strefa lokalna)
<code>today_at('06:30')</code>	dzisiejsza data o podanej godzinie
<code>as_timestamp(x)</code>	czas na sekundy epoki (do arytmetyki)
<code>timedelta(minutes=15)</code>	odcinek czasu do dodawania/odejmowania
<code>states.sensor.x.last_changed</code>	kiedy stan ostatnio się zmienił
<code>expand('group.czujniki')</code>	rozwińcie grupy do listy encji
<code>selectattr(...)</code> <code>list</code>	filtrowanie list (np. tylko encje w stanie on)

Dwa przykłady „z życia”, które łączą klocki:

```
{# Ile minut trwa bieżąca sesja ładowania? #}
{{ ((now() - states.sensor.ladowarka_status.last_changed).total_seconds() / 60) |
  round(0) }}

{# Ktore czujniki sa niedostepne? #}
{{ states.sensor | selectattr('state', 'in', ['unknown', 'unavailable'])
  | map(attribute='entity_id') | list | join(', ') }}
```

Wskazówka: W wyzwalaczu i warunku template obowiązuje subtelna zasada: wyzwalacz przelicza się tylko wtedy, gdy zmieni się któraś z encji *użytych w szablonie* (HA sam je wykrywa). Szablon bez żadnej encji — np. sam `now()` — nie będzie się odświeżał tak, jak intuicja podpowiada; do reguł czasowych służą wyzwalacze `time/time_pattern`.

9. Elementy pomocnicze: helpery, skrypty, sceny, blueprinty

9.1. Helpery — pamięć i pokrętła Twoich reguł

Helpery (**Ustawienia** → **Urządzenia i usługi** → **Pomocnicy**) to encje tworzone ręcznie, bez fizycznego urządzenia. To one czynią automatyzacje konfigurowalnymi i odpornymi na restart:

input_boolean

wirtualny przełącznik — flaga trybu („goście”, „wakacje”, „serwis”). Automatyzacje sprawdzają go w warunkach; Ty przełączasz z pulpitu.

input_number

liczba z suwakiem — progi i nastawy (cena prądu, minimalna nadwyżka PV). Zmieniasz próg bez edycji reguły.

input_select

lista opcji — tryby pracy („Eko / Komfort / Poza domem”).

input_datetime, input_text

godzina/data i tekst do wpisania.

timer

odliczanie z encją i zdarzeniem `timer.finished`; w przeciwieństwie do `delay` ma opcję przetrwania restartu (`restore`).

counter

licznik zdarzeń (ile sesji ładowania w tym tygodniu?).

schedule

tygodniowy grafik „włącz/wyłącz” rysowany myszką — encja jest `on` w zaznaczonych oknach. Wymarzony do taryf (przepis 10.1).

grupa, próg, pochodna, statystyki, Riemann

gotowe encje przetwarzające: grupowanie, binarne przekroczenie progu, pochodna (szybkość zmian), statystyki kroczące oraz całka Riemanna — najważniejsza z nich dla energetyki (rozdz. 12).

9.2. Skrypty i sceny

Skrypt to nazwana sekwencja akcji — wielokrotnego użytku, wywoływana z automatyzacji, pulpitu albo głosowo. Skrypt może przyjmować parametry (sekcja `fields`) i — od nowszych wydań — zwracać wynik (`response_variable`). Reguła kciuka: logika, którą chcesz wywołać z więcej niż jednego miejsca, idzie do skryptu; automatyzacje tylko decydują *kiedy*.

Scena to zapamiętany zestaw stanów (światła 30%, rolety w dół). `scene.turn_on` przywraca zestaw; sprytny wariant: `scene.create` robi „migawkę” bieżących stanów, a po chwili można do niej wrócić — np. mrugnij światłami na czerwono przy błędzie ładowarki, potem odtwórz, co było.

9.3. Blueprinty — automatyzacje z formularzem

Blueprint to szablon automatyzacji z polami do wypełnienia („wybierz czujnik ruchu”, „wybierz światło”). Z jednego blueprintu tworzy się dziesiątki instancji; społeczność publikuje setki gotowych (import: **Ustawienia** → **Automatyzacje i sceny** → **Blueprinty** → **Importuj**, wklejenie adresu URL). Dla firmy takiej jak AMPERE POINT blueprint to także sposób dystrybucji sprawdzonych reguł do klientów — jedna dopracowana „ładowarka po taniej taryfie” zamiast indywidualnego YAML u każdego.

10. Przepisy — kompletne automatyzacje z komentarzem

Wszystkie przepisy używają przykładowych identyfikatorów (`switch.ładowarka`, `sensor.ładowarka_status` itd.) — podmień je na własne z zakładki Encje.

Statusy ładowarek serii Q przyjmują wartości `charger_free`, `charger_insert`, `charger_wait`, `charger_charging`, `charger_pause`, `charger_end`, `charger_fault` (Q37: encja `x_work_state` o analogicznych stanach). Każdy przepis wklejysz w edytorze automatyzacji przez menu „...” → **Edytuj w YAML**.

10.1. Ładowanie w taniej taryfie — wersja dojrzała (helper schedule)

Zamiast wpisywać godziny w regułę, rysujemy grafik: pomocnik **schedule** o nazwie `schedule.tania_taryfa` (np. 22-6 w dni robocze, całe weekendy). Automatyzacja robi się trywialna, a zmiana taryfy to edycja grafiku, nie kodu:

```
alias: Ładowarka wg taniej taryfy
description: Włącza ładowanie w oknach grafiku, wyłącza poza nimi
mode: single
triggers:
  - trigger: state
    entity_id: schedule.tania_taryfa
    id: zmiana
actions:
  - choose:
    - conditions:
      - condition: state
        entity_id: schedule.tania_taryfa
        state: "on"
      sequence:
        - action: switch.turn_on
          target: { entity_id: switch.ladowarka }
    default:
      - action: switch.turn_off
        target: { entity_id: switch.ladowarka }
```

Komentarz: wyzwaczem jest *każda* zmiana stanu grafiku (wejście i wyjście z okna); `choose` z gałęzią domyślną załatwia obie strony jedną regułą. Chcesz wyjątek „ładuj natychmiast”? Dodaj `input_boolean.laduj_teraz` i warunek `off` na nim w gałęzi wyłączającej.

10.2. Powiadomienie o końcu ładowania — z danymi sesji

```
alias: Koniec ładowania - powiadomienie
mode: single
triggers:
  - trigger: state
    entity_id: sensor.ladowarka_status
    from: "charger_charging"
    to:
      - "charger_end"
      - "charger_free"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Ładowarka"
      message: >-
        Ładowanie zakończone. Sesja:
        {{ states('sensor.ladowarka_energia_sesji') | float(0) | round(2) }} kWh
        ({{ (states('sensor.ladowarka_energia_sesji') | float(0)
          * states('input_number.cena_kwh') | float(1)) | round(2) }} zł).
```

Komentarz: to: przyjmuje listę — łapiemy i naturalny koniec (charger_end), i wypięcie wtyczki. Koszt liczy szablon z pomocnika input_number.cena_kwh — cena prądu jest nastawą na pulpicie, nie liczbą zaszytą w regule.

10.3. Flagowy: ładowanie nadwyżką z fotowoltaiki z płynną regulacją prądu

Cel: gdy dom oddaje energię do sieci, ładowarka ma ją przechwycić — i *dopasowywać prąd* do bieżącej nadwyżki, zamiast tylko włączać się i wyłączać. Założenia: czujnik sensor.moc_eksportu (dodatni = oddajemy; z falownika po Modbus albo z naszego symulatora SUN2000), ładowarka trójfazowa z nastawą number.ladowarka_limit_pradu (6-16 A), histereza w pomocnikach: input_number.pv_prog_startu (np. 4200 W) i input_number.pv_prog_stopu (np. 800 W).

```
alias: Ładowanie nadwyżka PV (3 fazy, płynna regulacja)
description: >-
  Start powyżej progu startu, płynne dopasowanie prądu co minute,
  stop po spadku poniżej progu stopu utrzymanym 5 minut.
mode: single
triggers:
  - trigger: numeric_state
    entity_id: sensor.moc_eksportu
    above: input_number.pv_prog_startu
    for: "00:03:00"
    id: start
  - trigger: time_pattern
    minutes: "/1"
    id: koryguj
  - trigger: numeric_state
    entity_id: sensor.moc_eksportu
    below: input_number.pv_prog_stopu
    for: "00:05:00"
    id: stop
conditions:
  - condition: state
    entity_id: sensor.ladowarka_status
    state:
      - "charger_insert"
      - "charger_wait"
      - "charger_charging"
      - "charger_pause"
actions:
  - variables:
      eksport: "{{ states('sensor.moc_eksportu') | float(0) }}"
      pobor_ladowarki: "{{ states('sensor.ladowarka_moc') | float(0) }}"
      do_dyspozycji: "{{ eksport + pobor_ladowarki }}"
      prad: >-
        {{ [16, [6, (do_dyspozycji / (3 * 230)) | int] | max] | min }}
  - choose:
      - conditions: "{{ trigger.id == 'stop' }}"
        sequence:
          - action: switch.turn_off
            target: { entity_id: switch.ladowarka }
      - conditions: "{{ trigger.id == 'start' }}"
        sequence:
          - action: number.set_value
            target: { entity_id: number.ladowarka_limit_pradu }
            data: { value: "{{ prad }}" }
          - action: switch.turn_on
```

```

        target: { entity_id: switch.ladowarka }
    - conditions: >-
        {{ trigger.id == 'koryguj'
          and is_state('switch.ladowarka','on')
          and (prad - states('number.ladowarka_limit_pradu')|float(0)) | abs >= 1
        }}
    sequence:
    - action: number.set_value
      target: { entity_id: number.ladowarka_limit_pradu }
      data: { value: "{{ prad }}" }

```

Komentarz linia po linii, bo to najgęstszy przepis poradnika:

- **Trzy wyzwalacze z id:** start (nadwyżka utrzymana 3 min — chmura przelatująca nad domem nie wystartuje ładowania), korekta co minutę, stop (spadek utrzymany 5 min — auto nie „mruga”). Progi są encjami `input_number`, więc stroisz je z pulpitu.
- **Warunek wspólny:** pojazd w ogóle podłączony (lista akceptowanych stanów).
- **Kluczowa zmienna do dyspozycji:** do bieżącego eksportu *dodajemy* aktualny pobór ładowarki — bo gdy ładowarka już pracuje, jej pobór pomniejszył eksport; bez tej poprawki reguła „dusiłaby” sama siebie (najczęstszy błąd w tego typu automatyzacjach w internecie!).
- **Przeliczenie na prąd:** moc dzielona przez $3 \times 230\text{ V}$ (trzy fazy; dla ładowarki jednofazowej usuń „3 *”), obciąża do widełek 6-16 A zapisem „min-max” na listach Jinja2.
- **Korekta tylko przy różnicy $\geq 1\text{ A}$** — oszczędzamy urządzeniu ciągłego przestawiania o ułamki ampera.
- Tryb single: gdyby przebieg jeszcze trwał, kolejny tik minutowy po prostu przepadnie — tu to pożądane.

UWAGA: Minimalny prąd ładowania według normy IEC 61851 to 6 A — poniżej auto przerywa sesję. Dlatego widełki zaczynają się od 6 A, a start ma osobny, wyższy próg mocy: $6\text{ A} \times 3 \times 230\text{ V} \approx 4,1\text{ kW}$ nadwyżki musi być, żeby w ogóle ruszyć trójfazowo.

10.4. Alarm błędu z eskalacją — aż ktoś potwierdzi

Pomocnik: `input_boolean.alarm_potwierdzony`. Reguła powiadamia co 10 minut, maksymalnie 6 razy, dopóki ktoś nie przełączy potwierdzenia (z pulpitu albo przyciskiem w powiadomieniu — patrz następny przepis):

```

alias: Bład ładowarki - eskalacja
mode: single
triggers:
- trigger: state
  entity_id: sensor.ladowarka_status
  to: "charger_fault"
actions:
- action: input_boolean.turn_off
  target: { entity_id: input_boolean.alarm_potwierdzony }
- repeat:
  until:
    - condition: or
      conditions:
        - condition: state

```

```

        entity_id: input_boolean.alarm_potwierdzony
        state: "on"
      - "{{ repeat.index >= 6 }}"
    sequence:
      - action: notify.mobile_app_telefon
        data:
          title: "UWAGA: blad ladowarki ({{ repeat.index }}/6)"
          message: "Stan: {{ states('sensor.ladowarka_status') }}"
      - wait_for_trigger:
          - trigger: state
            entity_id: input_boolean.alarm_potwierdzony
            to: "on"
        timeout: "00:10:00"
        continue_on_timeout: true

```

Komentarz: `repeat.until` sprawdza warunek *po* każdym obiegu; `wait_for_trigger` z `timeout` realizuje „czekaj 10 minut albo do potwierdzenia — co pierwsze”. Zwróć uwagę na zerowanie flagi na początku: każdy nowy błąd zaczyna eskalację od czysta.

10.5. Powiadomienie z przyciskami (actionable) — pełny obieg

Dwie reguły. Pierwsza wysyła powiadomienie z przyciskiem; druga nasłuchuje zdarzenia, które aplikacja Companion odsyła po naciśnięciu:

```

alias: Auto podlaczone - zapytaj o start
mode: single
triggers:
  - trigger: state
    entity_id: sensor.ladowarka_status
    to: "charger_insert"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Ladowarka"
      message: "Auto podlaczone. Rozpoczac ladowanie?"
      data:
        actions:
          - action: "START_LADOWANIA"
            title: "Laduj teraz"
          - action: "POTWIERDZ_ALARM"
            title: "Potwierdz alarm"

```

```

alias: Obsługa przyciskow z powiadomien
mode: parallel
triggers:
  - trigger: event
    event_type: mobile_app_notification_action
    event_data: { action: "START_LADOWANIA" }
    id: laduj
  - trigger: event
    event_type: mobile_app_notification_action
    event_data: { action: "POTWIERDZ_ALARM" }
    id: potwierdz
actions:
  - choose:
      - conditions: "{{ trigger.id == 'laduj' }}"
        sequence:
          - action: switch.turn_on
            target: { entity_id: switch.ladowarka }
      - conditions: "{{ trigger.id == 'potwierdz' }}"

```

```
sequence:
  - action: input_boolean.turn_on
    target: { entity_id: input_boolean.alarm_potwierdzony }
```

Komentarz: przycisk nie wywołuje niczego bezpośrednio — odsyła *zdarzenie* na magistralę (rozdz. 2), a dopiero reguła-odbiornik decyduje, co ono znaczy. Tryb *parallel*: naciśnięcia z dwóch telefonów nie będą się blokować.

10.6. Watchdog niedostępności — system pilnuje sam siebie

```
alias: Watchdog - encje ładowarki niedostępne
mode: single
triggers:
  - trigger: state
    entity_id:
      - sensor.ladowarka_status
      - sensor.ladowarka_moc
    to:
      - "unavailable"
      - "unknown"
    for: "00:10:00"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "HA: utracono kontakt z ładowarką"
      message: >-
        {{ trigger.entity_id }} jest {{ trigger.to_state.state }}
        od 10 minut. Sprawdź internet / zasilanie / chmure Tuya.
```

Komentarz: *for* odsiewa sekundowe czknięcia chmury; zmienna *trigger* w szablonie mówi, *która* encja i w *jakim* stanie — jedna reguła obsługuje dziesiątki encji.

10.7. Raport dzienny o 21:00

Zakłada licznik dzienny `sensor.ladowarka_energia_dzienna` z pomocnika `utility_meter` (rozdz. 12):

```
alias: Raport dzienny ładowania
mode: single
triggers:
  - trigger: time
    at: "21:00:00"
conditions:
  - condition: numeric_state
    entity_id: sensor.ladowarka_energia_dzienna
    above: 0.1
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Raport ładowania - {{ now().strftime('%d.%m') }}"
      message: >-
        Dzis naładowano
        {{ states('sensor.ladowarka_energia_dzienna') | float(0) | round(1) }} kWh
        za ok. {{ (states('sensor.ladowarka_energia_dzienna') | float(0)
          * states('input_number.cena_kwh') | float(1)) | round(2) }} zł.
```

Komentarz: warunek „powyżej 0,1 kWh” wycisza raport w dni bez ładowania; format daty ustawia `strftime` — ten sam mechanizm przyda się w każdym powiadomieniu z datą.

11. Diagnostyka — gdy reguła nie robi tego, co miała

11.1. Ślady (traces) — najpotężniejsze i najmniej znane narzędzie

Każdy przebieg automatyzacji zostawia **ślad**: edytor reguły → zakładka **Ślady**. Zobacysz graf wykonania krok po kroku: który wyzwalacz strzelił, jak wypadły warunki, którą gałąź choose wybrano, jakie wartości miały zmienne — łącznie z pełnym obiektem trigger. HA trzyma domyślnie pięć ostatnich śladów na regułę. Dziewięć na dziesięć zagadek rozwiązuje się właśnie tutaj, bez zgadywania.

11.2. Reszta niezbędnika

- **Dziennik** (Logbook) — oś czasu „co się działo”: zmiany stanów i kto/co je wywołało.
- **Narzędzia deweloperskie** → **Stany** — podgląd i *ręczne ustawienie* dowolnego stanu (tylko do testów — integracja nadpisze przy najbliższym raporcie). Nieocenione do prowokowania reguł.
- **Narzędzia deweloperskie** → **Zdarzenia** — nasłuch magistrali (np. `mobile_app_notification_action` przy debugowaniu przycisków powiadomień).
- **Narzędzia deweloperskie** → **Szablon** — poligon Jinja2 na żywo.
- **Logi** (Ustawienia → System → Logi) z filtrem po nazwie integracji; poziom szczegółowości podnosi się usługą `logger.set_level` bez restartu.

11.3. Pułapki, które łapią każdego

- **Tryb single + długie czekanie** — kolejne wyzwolenia przepadają (rozd. 7).
- **for zeruje się przy każdej zmianie stanu** — czujnik migoczący wokół progu nigdy nie „uzbiera” czasu; pomaga histereza (dwa progi) albo pomocnik *próg* z wbudowaną histerezą.
- **Wyzwalacz progowy nie strzela „bo już jest powyżej”** — `numeric_state` reaguje na *przekroczenie*, nie na trwanie. Po restarcie HA nic nie „przekracza” — stany dopiero się ładują. Reguły krytyczne uzupełnij wyzwalaczem `homeassistant: start` plus warunkami.
- **unknown/unavailable w arytmetyce** — zawsze `| float(0) / has_value()` (rozd. 8); jedna niedostępna encja potrafi po cichu wyłączyć całą regułę na dobę.
- **Restart przerywa delay/wait** — długie odliczania przenoś do pomocnika `timer` z opcją `restore`.
- **Pętla sprzężenia** — reguła zmienia encję, którą sama jest wyzwalana. Ślady pokażą to od razu; rozwiązaniem bywa warunek odcinający albo rozdzielenie na dwie reguły.

12. Pomiary energii — od encji do panelu Energia

12.1. Co encja musi mieć, żeby panel ją przyjął

Panel Energia przyjmuje wyłącznie encje-liczniki: `device_class: energy`, jednostka kWh oraz `state_class: total_increasing` — czyli wartość *narastającą* (jak licznik w rozdzielnicy), która może się co najwyżej wyzerować (HA rozpoznaje zerowanie i policzy przyrost poprawnie). Chwilowa moc (W) ma `state_class: measurement` i do panelu jako źródło energii się nie nadaje — to najczęstsze nieporozumienie.

W ładowarkach serii Q rolę licznika pełni `forward_energy_total`. Gdy urządzenie daje tylko moc, energię wytwarza pomocnik **Integracja — suma Riemanna** (całkuje moc po czasie; wybierz metodę *lewostronną* — dla skokowych przebiegów mocy nie zawyża wyniku).

12.2. Taryfy i okresy: `utility_meter`

Pomocnik **`utility_meter`** to licznik źródłowy na okresy (dzienny, miesięczny — stąd `sensor.ladowarka_energia_dzienna` w przepisie raportu) i opcjonalnie na **taryfy**. Przy taryfach pomocnik tworzy encję wyboru; przełączanie taryfy to jedna automatyzacja sprzężona z grafiką z przepisu 10.1:

```
triggers:
- trigger: state
  entity_id: schedule.tania_taryfa
actions:
- action: select.select_option
  target: { entity_id: select.energia_ladowarki }
  data:
    option: >-
      {{ 'tania' if is_state('schedule.tania_taryfa','on') else 'droga' }}
```

Koszty w panelu Energia: cena stała, encja ceny (np. z pomocnika `input_number.cena_kwh`) albo cennik godzinowy z integracji giełdowej — panel przeliczy zużycie na złotówki za każdy okres.

13. Niezawodność, kopie zapasowe, dostęp zdalny

13.1. Kopie zapasowe

W HA OS: **Ustawienia** → **System** → **Kopie zapasowe** — od wydań 2025 z harmonogramem automatycznym i szyfrowaniem; kopię *pobieraj* poza urządzenie (karta SD, która umarła, zabiera też kopię na siebie). W wariantcie kontenerowym kopią jest folder `config` (nasz przewodnik, rozdz. o kopiach). Rytuał minimum: kopia przed każdą aktualizacją i raz w tygodniu automatycznie.

13.2. Aktualizacje bez dramatów

Czytaj noty wydania (sekcja *Breaking changes* — zmiany niekompatybilne); nie aktualizuj w dniu premiery, chyba że lubisz przygody — poprawkowe wydanie .1-.3 wychodzi zwykle w ciągu dni; po aktualizacji przejrzyj **Ustawienia** → **System**

→ **Naprawy** i logi. Dodatki społecznościowe (xtend_tuya!) sprawdzaj pod kątem zgodności *przed* dużym skokiem wersji rdzenia.

13.3. Dostęp spoza domu — trzy drogi

Home Assistant Cloud (Nabu Casa)

abonament: szyfrowany tunel bez konfiguracji, plus asystenci głosowi; przy okazji finansuje rozwój projektu. Najprostsze i bezpieczne.

VPN (Virtual Private Network — wirtualna sieć prywatna)

np. WireGuard albo Tailscale: telefon „wchodzi” do sieci domowej i widzi HA jak z kanapy. Darmowe, bardzo bezpieczne, wymaga kwadransa konfiguracji.

Wystawienie portu / reverse proxy

dla zaawansowanych: własna domena, certyfikaty TLS, obowiązkowo uwierzytelnianie dwuskładnikowe. Największa kontrola — i największa odpowiedzialność; nigdy nie wystawiaj HA „gołym” portem 8123 bez HTTPS.

Niezależnie od drogi: załóż osobne konta domownikom (bez uprawnień administratora), włącz uwierzytelnianie dwuskładnikowe na kontach administracyjnych, a do integracji zewnętrznych używaj *tokenów długoterminowych* zamiast hasła.

14. Najczęstsze problemy — ściągą

Objaw	Pierwszy ruch
Automatyzacja „czasem nie działa”	Zakładka Ślady: który wyzwalacz, które warunki. W 90% winne: tryb single albo warunek na encji unavailable.
Reguła progowa nie startuje po restarcie	numeric_state reaguje tylko na przekroczenie; dodaj wyzwalacz startu HA + warunki.
Powiadomienia nie przychodzą	Sprawdź nazwę usługi notify.mobile_app_* (Narzędzia deweloperskie → Akcje), zezwolenia aplikacji na telefonie, tryb oszczędzania baterii.
Przycisk z powiadomienia nic nie robi	Nasłuchaj zdarzenia mobile_app_notification_action w Narzędziach deweloperskich → Zdarzenia — literówka w action to klasyk.
Panel Energia nie widzi encji	Brak state_class: total_increasing / złej jednostki; użyj licznika (forward_energy_total), nie mocy.
Wartości energii „skaczą” po restarcie urządzenia	Licznik urządzenia się wyzerował — total_increasing to obsłuży; nie używaj total bez potrzeby.
Zigbee gubi urządzenia	Koordinator na przedłużaczu USB, z dala od routera i dysków USB 3.0; dołóż routery (wtyczki); rozważ kanał 15/20/25.

Objaw	Pierwszy ruch
Encje chmurowe co noc unavailable	Restart routera / przerwa u operatora / serwis chmury — watchdog z przepisów odsieje sekundowe czknięcia od realnych awarii.
Szablon działa w testerze, w regule nie	W testerze patrzysz na wynik, w wyzwalaczu liczy się <i>zmiana</i> wyniku i encje wykryte w szablonie; sprawdź ślad.
Po aktualizacji HA coś zniknęło	Ustawienia → System → Naprawy + noty wydania; dodatki HACS mogły wymagać własnej aktualizacji.

15. Słowniczek pojęć i skrótów

Akcja (action)

pojedynczy krok automatyzacji: wywołanie usługi, decyzja, pauza.

API interfejs programistyczny — „okienko podawcze” dla programów.

BLE / BTHome

energooszczędny Bluetooth / otwarty format rozgłoszeń czujników.

Blueprint

szablon automatyzacji z polami do wypełnienia.

Border router

urządzenie łączące siatkę Thread z siecią domową IP.

Broker

pośrednik MQTT rozdzielający wiadomości (np. Mosquitto).

Config entry (wpis konfiguracyjny)

zapisany wynik kreatora integracji; bez niego integracja nie działa.

DP (data point)

ponumerowana zmienna urządzenia Tuya (typ + skala).

Encja / urządzenie / obszar

wartość → teczka wartości → pomieszczenie.

ESPHome

system budowy własnych urządzeń na ESP32 z konfiguracji YAML.

Helper (pomocnik)

encja tworzona ręcznie: flaga, liczba, grafik, licznik, minutnik.

HTTP / REST / WebSocket

protokół stron www / styl API „pobierz-ustaw” / stałe łącze dwukierunkowe.

IoT class

deklaracja integracji: lokalnie czy chmurowo, push czy poll.

Jinja2

język szablonów HA ({{ ... }}).

LWT

„testament” MQTT — wiadomość publikowana przez brokera po zniknięciu urządzenia.

Magistrala zdarzeń (event bus)

wewnętrzna tablica ogłoszeń HA; wszystko jest zdarzeniem.

Matter / Thread

wspólny język urządzeń po IP / radiowa siatka IPv6 (IEEE 802.15.4).

mDNS / SSDP / DHCP

mechanizmy, którymi HA wykrywa urządzenia w sieci.

Mesh (siatka)

topologia, w której urządzenia przekazują pakiety dalej (Zigbee, Z-Wave, Thread).

Modbus (RTU/TCP)

przemysłowy protokół rejestrów 16-bitowych (RS-485 / port 502).

MQTT / QoS / retain

protokół publikuj-subskrybuj / poziom gwarancji doręczenia / wiadomość zatrzymana.

ONVIF / RTSP

standard zarządzania kamerami / protokół strumienia wideo.

OTA

aktualizacja urządzenia „przez powietrze”.

Recorder / SQLite

komponent zapisu historii / domyślna baza danych w jednym pliku.

Scena / skrypt

zapamiętany zestaw stanów / nazwana sekwencja akcji.

state_class (measurement / total_increasing)

klasa statystyk encji: wartość chwilowa / licznik narastający.

Ślad (trace)

zapis przebiegu automatyzacji krok po kroku.

Tryb (mode)

zachowanie reguły przy ponownym wyzwoleniu: single, restart, queued, parallel.

utility_meter

pomocnik tnący licznik na okresy i taryfy.

Warunek / wyzwalacz

filtr „czy działać” / zdarzenie budzące regułę.

Webhook

adres URL wyzwalający automatyzację żądaniem HTTP.

ZHA / Zigbee2MQTT

dwie drogi obsługi Zigbee w HA (wbudowana / przez MQTT).

Z-Wave JS

serwer i integracja obsługi Z-Wave.

Koniec dokumentu — wersja 2, 2026-07-07 (v2 = v1 + rozdział „klik po kliku” + wdrożenia symulacja→sprzęt w rozdz. 5 + paczka narzędzi ha_sim_tools). Kolejne wersje oznaczać v2, v3... bez usuwania poprzednich.

Dokumenty powiązane: przewodnik instalacji (v2), raport xtend_tuya (v1), poradnik
kliencki www (v2), pv_sim (symulator falownika).